Hackathon Finder

System Design Document

Version 1.0 | 14 Oct 2026 | Status: draft, ready to build | Working name: HackRadar

1. Overview and goals

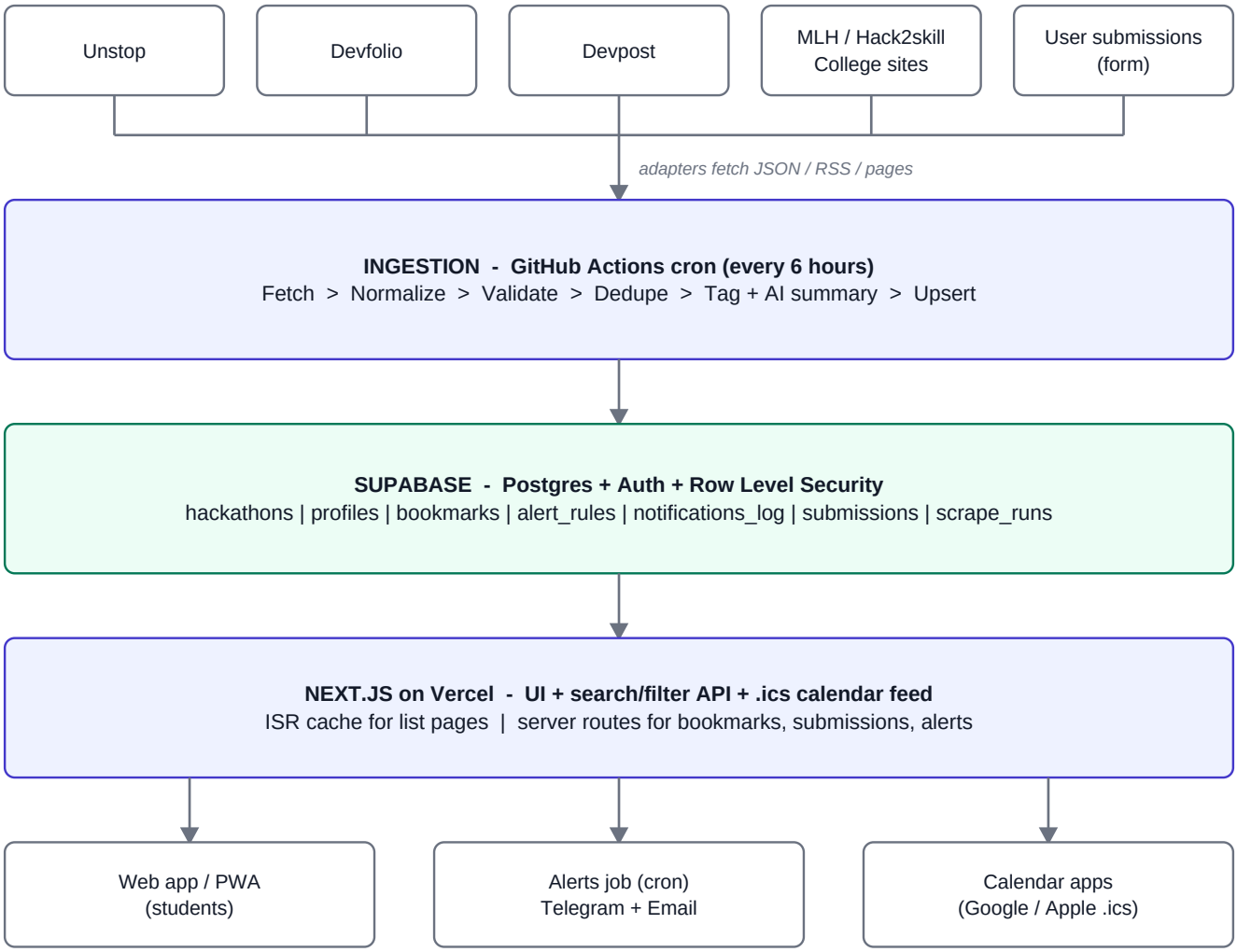
Hackathon Finder collects hackathon listings from many platforms and colleges, normalizes them into one format, and lets students search, filter and track them with deadline alerts. The system must run at zero hosting cost, be maintainable by one developer, and keep working when individual sources break.

- **Aggregate:** pull listings from multiple sources on a schedule, no manual copying.
- **Normalize:** one clean schema (dates, mode, city, prize, eligibility, tags) regardless of source.
- **Discover:** fast keyword search and filters, with closing-soon prioritised.
- **Notify:** Telegram and email alerts for new matches and approaching deadlines.
- **Crowdsource:** a moderated submit form for college and local events that big platforms miss.

2. Constraints and principles

- **Budget Rs 0:** only free tiers. No paid search service, queue or server.
- **Solo developer:** prefer managed services and few moving parts over clever infrastructure.
- **Scraping is fragile:** every source sits behind an adapter so one failure never takes down the rest.
- **Link back always:** store a short summary and metadata, send users to the original page to register.
- **Compute at read time:** status (open / closing soon / closed) is derived from dates, never stored.

3. High-level architecture



Data flows one way: sources, then ingestion, then the database, then the app, then users. The ingestion job is the only writer of listing data (plus approved submissions), which keeps the web app simple and read-heavy, and lets the cached pages serve most traffic without touching the database.

4. Components

Component	Technology	Responsibility	Why this choice
Web app + API	Next.js on Vercel Hobby	UI, search/filter endpoints, .ics feed, bookmark and submit routes	One codebase for frontend and backend; free; instant deploys from Git
Database + Auth	Supabase (Postgres)	Listings, users, bookmarks, alert rules, logs	Real SQL, full-text search, trigram matching, built-in auth and RLS
Ingestion	GitHub Actions cron + Node scripts	Fetch, normalize, dedupe, enrich, upsert	Free scheduled compute with no server to run; logs included
AI enrichment	Gemini API (free tier)	3-line summary, tag extraction, eligibility parsing	Free; run only on new or changed listings and cache the result
Alerts	Telegram Bot API + email (Resend or Brevo free)	Push new-match and closing-soon messages	Telegram is free and instant; email for people not on Telegram
Search	Postgres full-text + pg_trgm	Keyword search, typo tolerance, duplicate detection	No extra service needed at this data size

5. Data model

Core table: **hackathons**. Unique constraint on (source, source_id) makes ingestion idempotent.

Column	Type	Notes
id	uuid PK	generated
source / source_id	text / text	unique together, e.g. unstop / 12345
title, organizer	text	title is the main search field
description / summary	text / text	raw text, and the 3-line AI summary
mode	text	online | offline | hybrid
city, state	text	null for online events
start_date, end_date	timestamptz	event dates
reg_start, reg_deadline	timestamptz	registration window; drives status and alerts
prize_text / prize_amount_inr	text / int	text as shown, numeric when parsable
team_min, team_max	smallint	team size range
eligibility	jsonb	students_only, min_year, regions
tags	text[]	ai, web, iot, fintech, and similar
source_url, image_url	text	source_url is where users go to register
canonical_id	uuid null	points to the master row when duplicates are merged
verified, is_user_submitted	bool	moderation flags
created_at, updated_at, last_seen_at	timestamptz	last_seen_at lets us hide listings a source dropped

Other table	Key columns	Purpose
profiles	user_id, skills[], year, city, calendar_token	Personalisation and private calendar feed link
bookmarks	user_id, hackathon_id	Saved hackathons
alert_rules	user_id, filters jsonb, channel, telegram_chat_id, active	What each user wants to be told about
notifications_log	user_id, hackathon_id, kind, sent_at (unique on first three)	Prevents sending the same alert twice
submissions	id, payload jsonb, status, submitted_by, reviewed_at	Moderation queue for user-submitted events
scrape_runs	source, started_at, finished_at, fetched, inserted, updated, error	Health monitoring per source

6. Ingestion pipeline

Each source implements the same small contract, so adding a platform means writing one file.

```
// every source implements this
export interface SourceAdapter {
  name: string;                          // "unstop"
  fetchListings(): Promise<RawItem[]>;   // JSON endpoint, RSS or page
  normalize(raw: RawItem): Hackathon | null;  // null = skip bad row
}
```

Step	What happens
Fetch	Run all adapters in parallel with timeouts. Prefer official APIs or the JSON the site's own frontend loads; parse HTML only as a last resort.

Step	What happens
Normalize	Map each source's fields to the common schema. Parse dates to UTC, detect mode and city, extract prize amount and team size.
Validate	Require title, source_url and at least one date. Drop or flag rows that fail instead of inserting junk.
Dedupe	Normalize the title (lowercase, strip year, edition and punctuation), then match on trigram similarity above 0.8 plus start date within 2 days. Link duplicates through canonical_id.
Enrich	Only for new or changed rows: Gemini produces a short summary and tags. Cache by content hash.
Upsert	INSERT ... ON CONFLICT (source, source_id) DO UPDATE. Set last_seen_at = now().
Log	Write a scrape_runs row per source with counts and any error message.

Status logic (computed in queries and the UI): upcoming if now < reg_start; open if now < reg_deadline; closing soon if reg_deadline is within 72 hours; closed otherwise.

> **Failure handling:** each adapter runs in its own try/catch. A source that fails is retried once, then skipped and logged. Three failed runs in a row sends the owner a Telegram message. Listings not seen for 14 days are hidden rather than deleted.

7. Search and filter design

Postgres handles search at this scale (a few thousand rows). A generated tsvector column gives ranked keyword search, GIN indexes keep filters fast, and pg_trgm handles typos and duplicate detection.

```
create extension if not exists pg_trgm;

alter table hackathons add column fts tsvector generated always as (
  setweight(to_tsvector('english', coalesce(title,'')), 'A') ||
  setweight(to_tsvector('english', coalesce(organizer,'')), 'B') ||
  setweight(to_tsvector('english', coalesce(summary,'')), 'C')
) stored;

create index hackathons_fts_idx      on hackathons using gin (fts);
create index hackathons_tags_idx     on hackathons using gin (tags);
create index hackathons_deadline_idx on hackathons (reg_deadline);

-- list query
select * from hackathons
where (:q is null or fts @@ websearch_to_tsquery('english', :q))
  and (:mode is null or mode = :mode)
  and (:city is null or city ilike :city)
  and (:tags is null or tags && :tags)
  and reg_deadline > now()
order by reg_deadline asc
limit 20 offset :offset;
```

8. API design

Endpoint	Method	Purpose
/api/hackathons	GET	List with q, mode, city, status, tag, prize_min, team_max, sort, page
/api/hackathons/:id	GET	Detail card with summary, dates, eligibility and source link
/api/bookmarks	POST / DELETE	Save or remove a bookmark (auth required)
/api/alerts	GET / POST / DELETE	Manage alert rules
/api/submissions	POST	Submit a hackathon (rate limited)
/api/calendar/:token.ics	GET	Private calendar feed of the user's bookmarks
/api/telegram	POST	Bot webhook: /start, /link, /mute commands
/api/admin/submissions/:id	PATCH	Approve or reject (admin only)

9. Alerts subsystem

- An hourly job selects hackathons created in the last hour and those whose deadline crosses 48 hours and 24 hours.
- Each is matched against active alert_rules (tags, mode, city, prize threshold).
- notifications_log (unique on user, hackathon, kind) guarantees one message per event per user.
- Delivery: Telegram first, email if enabled. Weekly digest is a separate low-frequency job.

10. Recommendation (match score)

A simple, explainable heuristic, tuned later with real usage:

```
score = 100 * ( 0.45 * skill_overlap      # tags shared with the user's skills
              + 0.20 * eligibility_fit    # student, year, region all satisfied
              + 0.20 * deadline_urgency   # higher when closing soon but still open
              + 0.15 * prize_norm )       # prize relative to the max in the list
```

11. Free-tier budget and capacity

Service	Used for	Free allowance (approx.)	Expected use
Vercel Hobby	Hosting, API routes	Generous bandwidth; non-commercial use	Far below limits with ISR caching
Supabase Free	Postgres, Auth	About 500 MB database; project pauses after a week idle	A few MB for thousands of rows; a daily ping prevents pausing
GitHub Actions	Cron ingestion and alerts	Unlimited on public repos; about 2,000 min/month on private	4 runs/day x ~3 min = ~360 min/month
Gemini API	Summaries and tags	Rate-limited free tier	Only new or changed rows, cached
Telegram Bot API	Alerts	Free	Negligible
Resend / Brevo	Email	Small daily free quota	Digest and opt-in only
Domain	Optional	Use the free .vercel.app address	Rs 0

Free-tier limits change, so confirm each figure on the provider's pricing page before launch. If the project is monetised, Vercel Hobby's non-commercial rule means moving to a paid plan.

12. Security, privacy and legal

- Row Level Security on user tables so people only read and write their own bookmarks and alert rules.
- Secrets (Supabase service key, Gemini key, bot token) live in Vercel and GitHub encrypted secrets, never in the repo.
- Rate limit the submit form and add a honeypot field to deter spam.
- Store only the minimum user data (email, skills, city). No phone numbers unless needed for alerts.
- Check each source's robots.txt and terms. Prefer official APIs. Store metadata and short summaries, not full copies, and always link back.
- Calendar feed URLs use an unguessable per-user token that can be regenerated.

13. Deployment, CI/CD and observability

- Git push to a feature branch creates a Vercel preview URL. Merge to main deploys to production automatically.
- Database changes are versioned SQL migration files applied through the Supabase CLI.
- Observability: scrape_runs table shows per-source health; Vercel logs cover the app; a tiny status page can read scrape_runs.

14. Risks and mitigations

Risk	Impact	Mitigation
A source changes its format or blocks scraping	Missing listings	Adapter isolation, alert after 3 failures, multiple sources, user submissions
Terms of service restrict scraping	Legal or takedown risk	Official APIs first, link back, store minimal data, be ready to drop a source
Bad parsing (wrong dates or deadlines)	Users miss real deadlines	Validation rules, show source link prominently, quick report-an-error button
Supabase pauses on inactivity	Site down	Daily keep-alive query from the cron job
Free-tier limits exceeded	Throttling	ISR caching, batch AI calls, monitor usage monthly
Spam submissions	Low-quality data	Moderation queue, rate limits, honeypot

15. Scaling path (if it grows)

Move Postgres search to a dedicated service only if queries slow down; add a job queue if ingestion outgrows a single cron run; add a CDN-cached public JSON feed for partners. None of these are needed for V1.